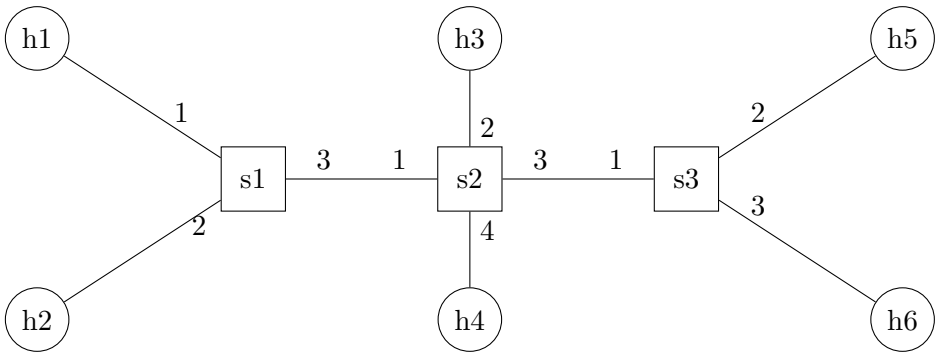
# Piattaforme Software per la Rete (Mod. 1) – 21/07/2017

**Esercizio 1**

Si consideri la rete in figura. I nodi h1,..., h$n$ contengono tabelle ARP prepopolate per tutta la rete. Gli indirizzi MAC e IP dei nodi h1,..., h$n$ sono MAC1,..., MAC$n$ e IP1,..., IP$n$. Sui nodi h1,..., h$n$ sono in esecuzione processi in ascolto su tutte le porte TCP e UDP di interesse. Il controllore Ryu è collegato ai nodi s1,..., s$m$ ed è configurato come indicato nel Programma 1. Le interfacce tra gli switch e il controllore non sono indicate in figura. Se non altrimenti specificato, il TTL iniziale è 200.




(a) Scrivere il contenuto delle tabelle openflow dopo l'accensione di tutti i nodi.

L'host h2, porta 7843 invia un datagramma UDP a h6, porta 543 con TTL iniziale 10.

(b) Scrivere tutti i pacchetti uscenti da tutte le interfacce di s1, s2, 3.

L'host h1, porta 3456 apre una connessione verso h5, porta 123.

(c) Scrivere tutti i pacchetti uscenti da tutte le interfacce di s1, s2, s3.

(d) L'host h3 risponde. Scrivere tutti i pacchetti uscenti da tutte le interfacce di s1, s2, s3.

(e) L'host h1 riscontra la risposta. Scrivere tutti i pacchetti uscenti da tutte le interfacce di s1, s2, s3.

(f) Scrivere il contenuto finale delle tabelle openflow.

**Programma 1 (1/2) – Switch Features**

```
@set_ev_cls(ofp_event.EventOFPSwitchFeatures, CONFIG_DISPATCHER)
def switch_features_handler(self, ev):
    datapath = ev.msg.datapath
    ofproto = datapath.ofproto
    parser = datapath.ofproto_parser

    match = parser.OFPMatch()
    actions = [parser.OFPActionOutput(ofproto.OFPP_CONTROLLER,
                                      ofproto.OFPCML_NO_BUFFER)]
    inst = [parser.OFPInstructionActions(ofproto.OFPIT_APPLY_ACTIONS,actions)]
    mod = parser.OFPFlowMod(datapath=datapath, priority=0,
                            match=match, instructions=inst)
    datapath.send_msg(mod)

    if (datapath.id == 2):
        match = parser.OFPMatch(in_port=1, eth_type=MPLS)
        actions = [ parser.OFPActionOutput(3) ]
        inst = [parser.OFPInstructionActions(ofproto.OFPIT_APPLY_ACTIONS,actions)]
        mod = parser.OFPFlowMod(datapath=datapath, priority=1,
                                match=match, instructions=inst)
        datapath.send_msg(mod)

    if (datapath.id == 3):
        match = parser.OFPMatch(eth_type=MPLS)
        actions = [ parser.OFPActionPopMpls(),
                    parser.OFPActionOutput(ofproto.OFPP_FLOOD) ]
        inst = [parser.OFPInstructionActions(ofproto.OFPIT_APPLY_ACTIONS,actions)]
        mod = parser.OFPFlowMod(datapath=datapath, priority=1,
                                match=match, instructions=inst)
        datapath.send_msg(mod)
```

**Programma 1 (2/2) – Packet In**

```
@set_ev_cls(ofp_event.EventOFPPacketIn, MAIN_DISPATCHER)
def packet_in_handler(self, ev):
    msg = ev.msg; datapath = msg.datapath
    ofproto = datapath.ofproto; parser = datapath.ofproto_parser
    in_port = msg.match['in_port']
    pkt = packet.Packet(msg.data)
    pkt_eth = pkt.get_protocol(eth.eth)
    pkt_ipv4 = pkt.get_protocol(ipv4.ipv4)
    pkt_udp = pkt.get_protocol(udp.udp)
    pkt_tcp = pkt.get_protocol(tcp.tcp)
    data = msg.data

    if (datapath.id == 1) and (pkt_tcp is not None) and (pkt_ipv4.dst==IP5):
        match = parser.OFPMatch(
            eth_dst=pkt_eth.src,eth_src=pkt_eth.dst,eth_type=IPv4,ip_proto=TCP)
        actions = [
            parser.OFPActionPushMpls(),
            parser.OFPActionSetField(mpls_label=1000),
            parser.OFPActionOutput(3)
        ]
        inst = [parser.OFPInstructionActions(ofproto.OFPIT_APPLY_ACTIONS,actions)]
        mod = parser.OFPFlowMod(datapath=datapath, priority=2,
                                match=match, instructions=inst)
        datapath.send_msg(mod)
    elif (pkt_udp is not None):
        if ( pkt_ipv4.ttl <= 5):
            return
        pkt_ipv4.ttl = pkt_ipv4.ttl - 5
        pkt_udp.csum = 0 # azzera checksum esistente
        pkt.serialize()  # calcola nuovi checksum
        data = pkt.data

    if (datapath.id == 1) and (in_port < 3):
        outport = 3
    elif (datapath.id == 3) and (in_port > 1):
        outport = 1
    else:
        outport = ofproto.OFPP_FLOOD
    actions = [parser.OFPActionOutput(outport)]
    out = parser.OFPPacketOut(datapath=datapath, buffer_id=msg.buffer_id,
                              in_port=in_port, actions=actions, data=data)
    datapath.send_msg(out)
```

**Esercizio 2**

Si consideri un sistema di misura del traffico in un router. Ad ogni pacchetto entranti è associato un identificativo di flusso $x$ ottenuto concatenando alcuni campi del pacchetto. La lunghezza massima dei pacchetti è 1024 byte.

Sull'identificativo $x$ sono calcolati $n$ hash usando la famiglia di funzioni indipendenti $h_i(x)$ con $1 \leq i \leq n$. Tali hash sono usati come indici in $n$ array distinti $A_1, \ldots, A_n$ di $m$ elementi ciascuno. Ad ogni pacchetto arrivato si incrementano di $L$ gli elemento $A_i[h_i(x)]$ per ogni $i$, dove $L$ è la lunghezza del pacchetto.

Se $A_i[h_i(x)] \geq S$ per ogni $i$, l'indentificativo $x$ viene salvato su disco.

Tutti gli array sono azzerati ogni $T = 1\,\text{s}$.

(a) Considerando quattro link entranti con capacità $C = 1\,\text{Gbit/s}$ ciascuno, dire quanti bit occorrono per rappresentare gli $n$ array.

(b) Dire quali campi occorre scegliere per rappresentare l'identificativo del flusso per avere nello stesso flusso tutti i pacchetti della stessa connessione TCP in entrambe le direzioni.

(c) Dire come scegliere la soglia $S$ in modo che si scriva su disco l'identificativo di tutti i flussi di almeno 50 Mbit/s. Ci sono falsi positivi? Ci sono falsi negativi?

(d) Dall'inizia della misura si è osservato un solo flusso $y$ da 75 Mbit/s. Considerando $m = 2^7$ e $n = 4$, dire qual è la probabilità che il primo pacchetto di un nuovo flusso sia copiato su disco.

**Esercizio 3**

Si consideri il seguente set di regole, per il quale implementare un meccanismo di Bit Vector Linear Search:

| Source Address | Destination Address | Source Port | Destination Port | Transport Protocol |
|---|---|---|---|---|
| 112.0.0.0/6 | * | * | 8080 | TCP |
| 104.0.0.0/8 | 32.0.0.0/3 | 80 | * | TCP |
| * | 104.0.0.0/8 | * | 53 | * |
| 104.0.0.0/6 | * | * | 123 | UDP |
| 32.0.0.0/3 | 104.0.0.0/8 | 23 | * | UDP |
| * | 104.0.0.0/6 | 25 | 80 | * |

(a) Definire il bitvector set per ciascun campo.

(b) L'algoritmo di ricerca del prefisso IP sorgente (prima colonna) è implementato usando un multibit trie con passo 3. Disegnare il trie.

(c) Si considerino il pacchetto (213.10.0.1, 81.10.0.2, 80, 123, UDP). Elencare le maschere identificate per ogni campo e identificare gli eventuali match.

**Esercizio 4**

Una matrice di commutazione crossbar di uno switch $4 \times 4$ usa un algoritmo per lo scheduling delle trasmissioni. Si assumano pacchetti tutti di lunghezza $L$,

Al tempo $t = 0$ sono presenti i seguenti pacchetti, in ordine dal più recente al più vecchio. Ogni pacchetto è etichettato con il numero dell'interfaccia di uscita cui è diretto.

**Ingresso 1:** 4, 2, 1

**Ingresso 2:** 1, 3, 2

**Ingresso 3:** –, 4, 3

**Ingresso 4:** –, 4, 1

(a) Svolgere l'algoritmo Take-a-Ticket (TaT) fino allo svuotamento di tutte le code, specificando ad ogni round quali pacchetti sono trasmessi e quali ticket sono assegnati. L'algoritmo di assegnazione dei ticket privilegia sempre l'interfaccia di ingresso con l'identificativo più piccolo.

(b) Si consideri il caso in cui i pacchetti diretti alla coda 2 abbiano lunghezza $2L$. Ripetere la simulazione dell'algoritmo TaT.

(c) Facendo riferimento all'algoritmo TaT con assegnazione casuale, si consideri uno scenario di traffico uniforme e code sempre sature. Qual è la probabilità che tre ingressi su quattro siano bloccati?